

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work nr. 9

Watching System Calls with strace

Course: Operating Systems (Track A: Linux)

Done by:
st. gr. FAF-243, Oberst Eduard

Checked by:
Patricia Reitman, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Purpose of the work

The purpose of this laboratory work is to watch, from the outside, the door between a program and the kernel: the system call. strace was used to read the system calls made by a real command (ls), then a small C program was written that calls open, write, read and close directly, and its system calls were watched with strace as well.

Preparation

All work was done directly on my own Ubuntu/Debian Linux laptop (no xv6, no QEMU — this is Track A). strace was already installed (version 6.19, checked with strace -V). The terminal session was recorded with script into lab9-transcript.log.

Part 1. See the calls of a real command

strace -V confirmed strace 6.19 was already installed. The following command was then run to watch `ls -l` from the outside:

```
$ strace -f -e trace=openat,read,write,close ls -l
```

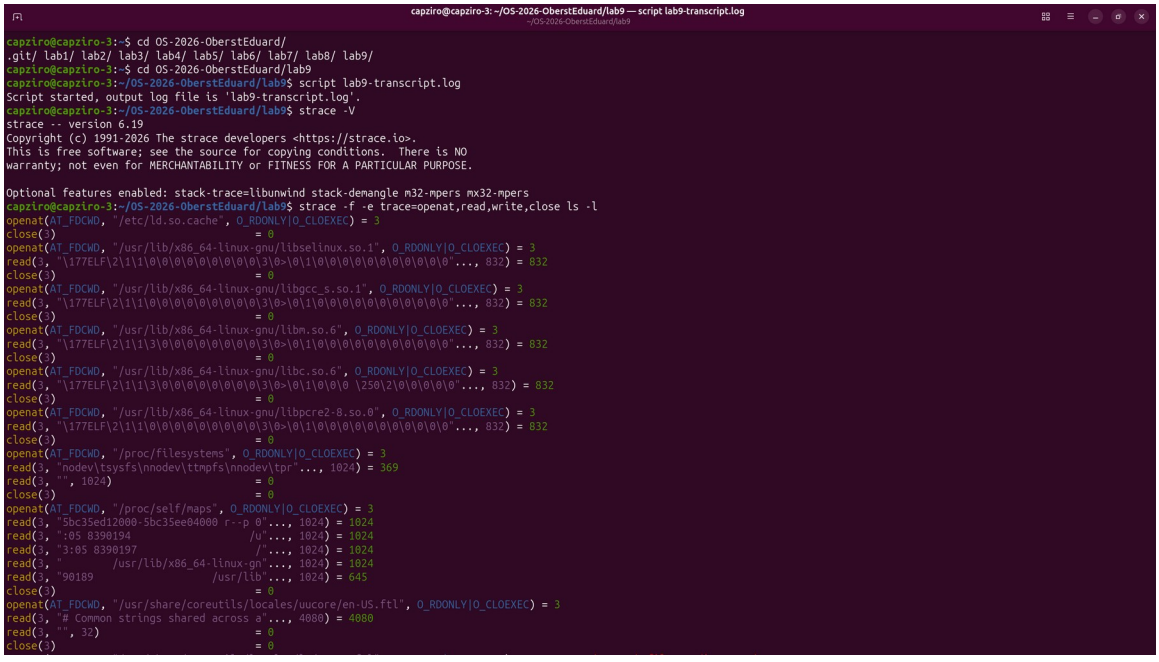


Figure 1.1 – strace -V and the start of the trace: the dynamic linker opening and reading ls's shared libraries

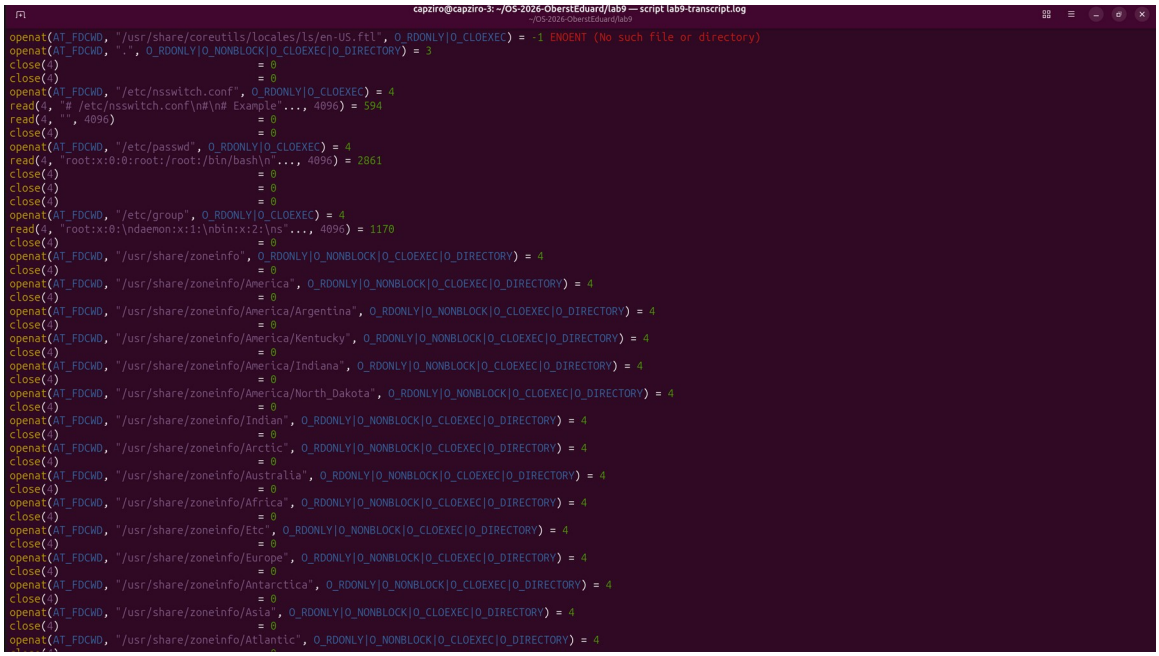


Figure 1.2 – the trace continues: reading /proc and /etc files and walking /usr/share/zoneinfo to resolve the local timezone

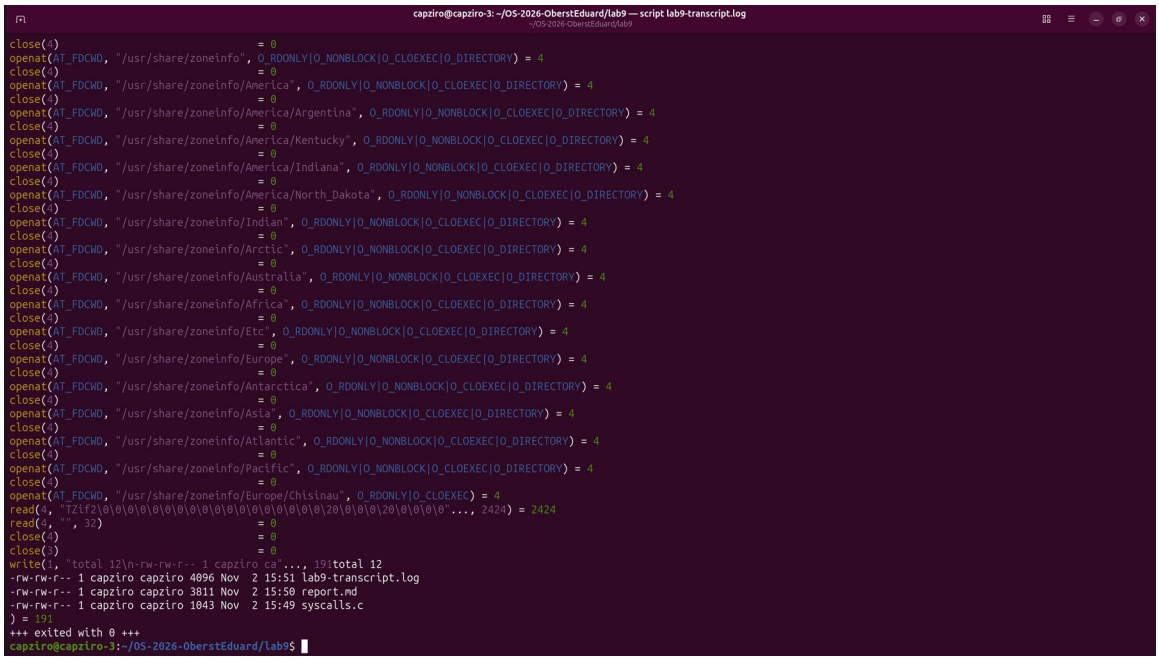


Figure 1.3 – the end of the trace: a single write(1, ...) prints the whole listing, then the process exits

Dozens of openat/read/close calls were needed just so ls could load its libraries and look up the files, timestamps and timezone, against a single write call that actually produced the visible output.

Observe

1. Which system call does ls use to print to the screen? To which descriptor does it write?

ls uses the write() system call to print to the screen, writing to file descriptor 1, which is standard output (stdout). In the strace trace this shows up as write(1, "total 12...", 191) = 191 in Figure 1.3, right after all the setup calls and just before the process exits.

Part 2. A program that calls the kernel directly

The file `syscalls.c` was created with the code from the lab (open, write, read, close — no printf is used to touch the file itself), then compiled and run:

```
$ gcc -Wall -Wextra -o syscalls syscalls.c
$ ./syscalls
```

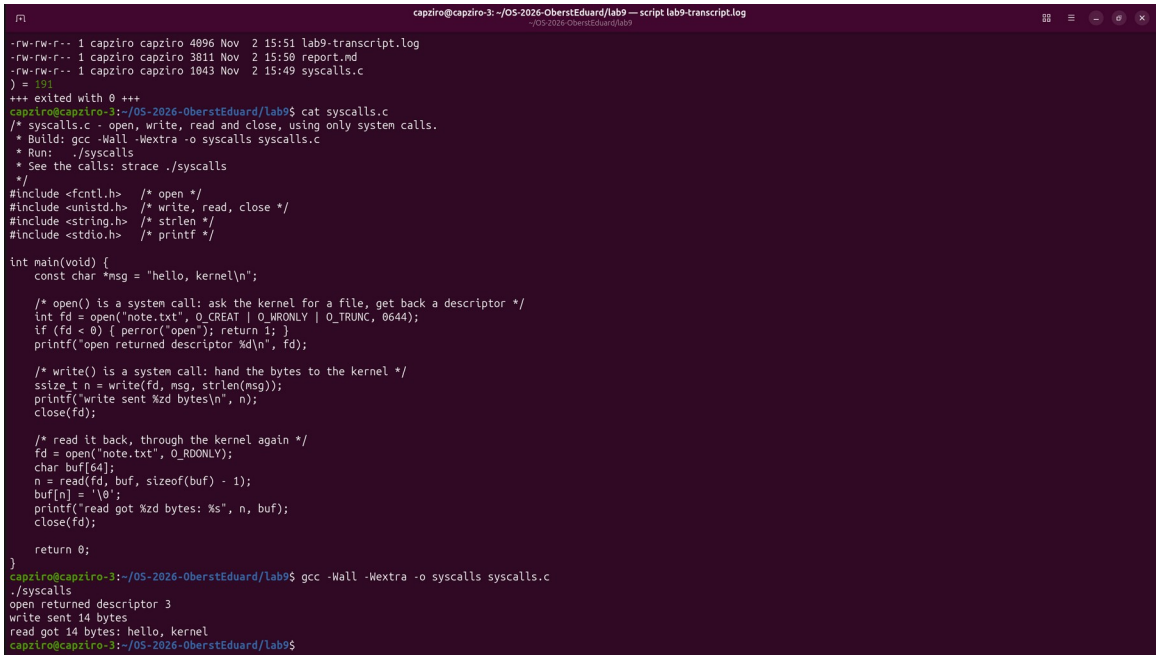


Figure 2.1 – cat syscalls.c showing the source, followed by the build and the program's own output

The output matched the expected output exactly: open returned descriptor 3, write sent 14 bytes, read got 14 bytes: hello, kernel. The first descriptor handed out by open() was 3, not 0.

The same program was then watched from outside with strace:

```
$ strace -e trace=openat,write,read,close ./syscalls
```

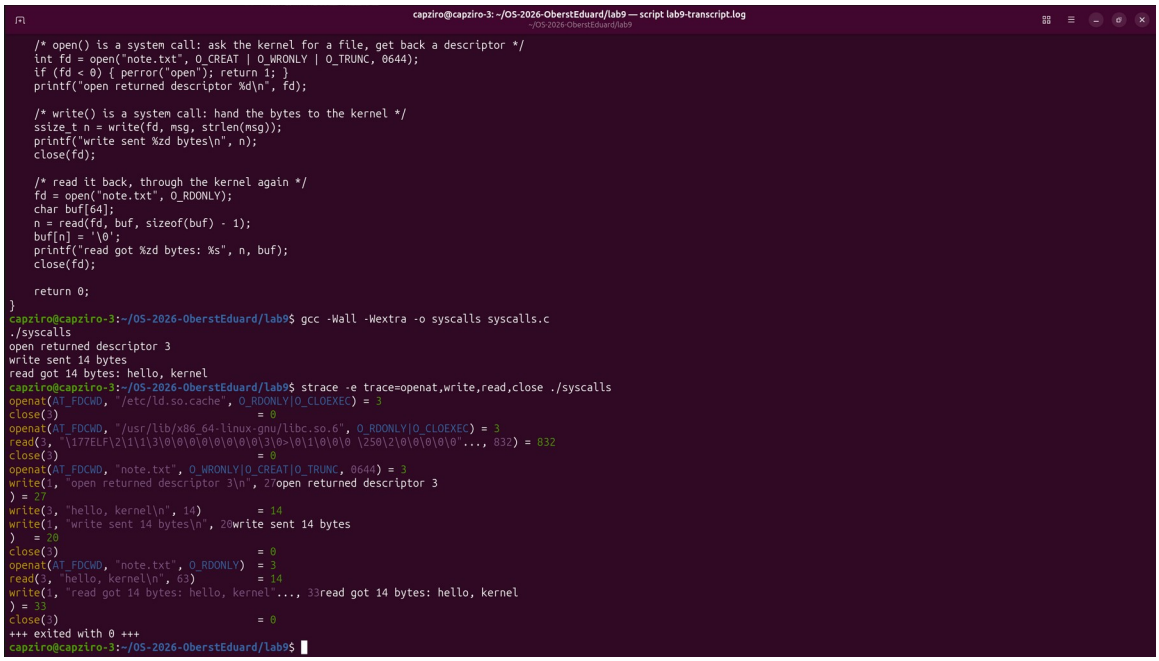


Figure 2.2 – the three C calls (open, write, read, close) appear as real system calls on note.txt

In the captured trace the three printf lines of syscalls.c itself also show up as write(1, "open returned descriptor 3\n", 27) = 27 and similar — proof that printf, just like the program's direct calls, ends up as a write() to descriptor 1 under the hood, exactly like ls did in Part 1.

Observe

1. The descriptor returned by the first openat is 3, not 0, 1 or 2. Why are those three already taken?

Every process receives three file descriptors already open when it starts, inherited from its parent shell: 0 is standard input, 1 is standard output, and 2 is standard error. The kernel always hands out the lowest-numbered descriptor that is free, so the first file the program opens itself is given the next free number, 3 — not because of anything special in the code, but simply because 0, 1 and 2 were already in use before main() even started.

Conclusions

strace makes the boundary between a program and the kernel visible: every line of output is a request the program made to the operating system and the value the kernel gave back. Watching ls showed that even a "simple" command makes many openat/read/close calls before it ever prints anything, and that printing itself is just a write to descriptor 1. Writing syscalls.c showed the same thing from the inside: the C library functions open, write, read and close are not special — they turn almost directly into system calls with the same name, and the "invisible" descriptors 0, 1 and 2 are already reserved before any file is opened by the program itself.

Submitted files: `syscalls.c`, `lab9-transcript.log` and this report.